UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

CARRERA PROFESIONAL DE INGENIERÍA INFORMÁTICA Y DE SISTEMAS

GUÍA DE LABORATORIO N.º 08

FFT, polinomios y transformadas sobre campos finitos

Asignatura:	Algoritmos Avanzados
Unidad:	II. Fundamentos algebraicos y transformadas rápidas
Semestre académico:	2026-II
Duración:	120 minutos
Organización:	Trabajo grupal en los equipos del proyecto semestral
Integrantes:	Aproximadamente cuatro estudiantes por equipo
Lenguaje:	Python 3
Docente:	Héctor Eduardo Ugarte Rojas

Equipo: _____

Integrantes: _____

Fecha: _____

Índice

1. Objetivo de la práctica **2**

2. Trabajo preparatorio obligatorio **2**

3. Materiales **2**

4. Marco teórico breve **3**

4.1. Convolución y multiplicación de polinomios . . . 3

4.2. DFT, FFT e IFFT . . . 3

4.3. Zero padding . . . 3

4.4. NTT sobre campos finitos . . . 3

5. Especificación mínima de la implementación **3**

6. Propiedades que deben comprobarse **4**

7. Distribución sugerida de los 120 minutos **4**

8. Ejercicios resueltos **5**

8.1. Ejercicio resuelto 1: convolución directa . . . 5

8.2. Ejercicio resuelto 2: FFT, IFFT y multiplicación . . . 5

8.3. Ejercicio resuelto 3: NTT e INTT en $\mathbb{F}_{17}$ . . . 6

9. Ejercicios propuestos **7**

9.1. Ejercicio propuesto 1: biblioteca integrada de FFT y NTT . . . 7

9.2. Ejercicio propuesto 2: pruebas diferenciales y análisis experimental . . . 8

10. Metodología de medición **8**

11. Resultados mínimos requeridos **9**

12. Preguntas de análisis **9**

13. Producto de entrega **10**

14. Tabla de evaluación **10**

15. Casos mínimos de prueba **11**

16. Referencias **11**

1. Objetivo de la práctica

Objetivo

Implementar y validar la multiplicación de polinomios mediante convolución directa, FFT compleja y NTT modular; comprobar la relación entre evaluación, producto punto a punto e interpolación; y analizar experimentalmente la complejidad, exactitud numérica y condiciones de uso de cada alternativa.

Al finalizar la práctica, el equipo será capaz de:

1. representar polinomios mediante vectores de coeficientes;
2. implementar la convolución directa y verificar sus coeficientes;
3. implementar FFT e IFFT radix-2;
4. aplicar zero padding con la longitud mínima correcta;
5. multiplicar polinomios mediante FFT y controlar el error de redondeo;
6. implementar una NTT e INTT sobre un campo finito compatible;
7. comparar corrección, número de operaciones y tiempo de ejecución;
8. explicar cuándo conviene usar FFT compleja o NTT modular.

2. Trabajo preparatorio obligatorio

Cada integrante deberá revisar:

1. multiplicación tradicional de polinomios;
2. convolución lineal de vectores;
3. números complejos y raíces n-ésimas de la unidad;
4. DFT e IDFT;
5. estrategia divide y vencerás de Cooley–Tukey;
6. inversión de bits y mariposas radix-2;
7. aritmética modular, inversos y orden de un elemento;
8. condición $N \mid (p-1)$ para una NTT de longitud N en $\mathbb{F}_p$.

3. Materiales

- Computadora con Python 3.11 o posterior, recomendado.
- Editor o IDE, terminal y Git.
- Biblioteca estándar: `cmath`, `math`, `random`, `time`, `statistics` y `csv`.
- Opcional para validación: `numpy` y `matplotlib`.
- Repositorio del equipo con la estructura solicitada.
- Diapositivas del tema 8 y casos de prueba preparados por el equipo.

4. Marco teórico breve

4.1. Convolución y multiplicación de polinomios

Si:

$$A(x) = \sum_{i=0}^{m-1} a_i x^i, \qquad B(x) = \sum_{j=0}^{n-1} b_j x^j,$$

entonces $C(x) = A(x)B(x)$ tiene coeficientes:

$$\boxed{c_k = \sum_{i+j=k} a_i b_j}.$$

El método directo usa $\Theta(mn)$ productos.

4.2. DFT, FFT e IFFT

La DFT evalúa el polinomio en raíces de unidad:

$$A_k = \sum_{j=0}^{N-1} a_j \omega_N^{jk}.$$

Cooley–Tukey separa índices pares e impares y obtiene:

$$A_k = E_k + \omega_N^k O_k, \qquad A_{k+N/2} = E_k - \omega_N^k O_k.$$

Esto produce la recurrencia:

$$T(N) = 2T(N/2) + \Theta(N) = \Theta(N \log N).$$

4.3. Zero padding

Si las entradas tienen m y n coeficientes, el producto tiene $m + n - 1$. Para una FFT radix-2 se utiliza:

$$N = 2^{\lceil \log_2(m+n-1) \rceil}.$$

Usar una longitud menor produce convolución circular y mezcla coeficientes.

4.4. NTT sobre campos finitos

La NTT reemplaza las raíces complejas por una raíz primitiva N-ésima ω en $\mathbb{F}_p$:

$$\omega^N \equiv 1 \pmod{p}, \qquad \omega^k \not\equiv 1 \pmod{p} \quad (1 \le k < N).$$

La condición necesaria es $N \mid (p-1)$. La transformada inversa usa ω^{-1} y multiplica cada salida por N^{-1} mód p.

5. Especificación mínima de la implementación

Listing 1: Interfaz requerida

```
def naive_convolution(a, b): ...
def next_power_of_two(value): ...
def fft(values): ...
```

```python
def ifft(values): ...
def multiply_fft(a, b, tolerance=1e-7): ...
def modular_inverse(a, modulus): ...
def ntt(values, invert, modulus, root): ...
def multiply_ntt(a, b, modulus, primitive_root): ...
def validate_root(root, length, modulus): ...
```

Convenciones

Los vectores almacenarán los coeficientes en orden ascendente: $(a_0, a_1, \ldots)$. Las operaciones de multiplicación devolverán exactamente $m + n - 1$ coeficientes. La FFT compleja normalizará resultados cercanos a enteros; la NTT devolverá residuos en $\{0, \ldots, p - 1\}$.

6. Propiedades que deben comprobarse

Después de cada operación de prueba, el equipo deberá verificar:

1. la salida tiene longitud $m + n - 1$;
2. la convolución directa coincide con la multiplicación manual;
3. `ifft(fft(a))` recupera a dentro de una tolerancia establecida;
4. la longitud elegida es potencia de dos y satisface $N \geq m + n - 1$;
5. `multiply_fft` coincide con la convolución directa después del redondeo;
6. la raíz NTT satisface el orden exacto requerido;
7. la INTT recupera el vector original módulo p;
8. `multiply_ntt` coincide con la convolución directa módulo p;
9. no aparecen coeficientes espurios por padding insuficiente;
10. las mediciones excluyen generación de datos, impresión y escritura de archivos.

7. Distribución sugerida de los 120 minutos

Etapa	Actividad	Tiempo
Inicio	Revisión de interfaces, convenciones e invariantes	10 min
Resuelto 1	Convolución directa y verificación	18 min
Resuelto 2	FFT, IFFT y multiplicación compleja	25 min
Resuelto 3	NTT e INTT en $\mathbb{F}_{17}$	22 min
Propuesto 1	Biblioteca integrada FFT/NTT	25 min
Propuesto 2	Pruebas diferenciales y experimentos	15 min
Cierre	Verificación individual y evidencias	5 min

8. Ejercicios resueltos

8.1. Ejercicio resuelto 1: convolución directa

Implemente la multiplicación tradicional de polinomios. Para cada par (i, j) se acumula $a_i b_j$ en la posición $i + j$.

Listing 2: Convolución lineal directa

```python
def naive_convolution(a, b):
    if not a or not b:
        return []

    result = [0] * (len(a) + len(b) - 1)
    for i, ai in enumerate(a):
        for j, bj in enumerate(b):
            result[i + j] += ai * bj
    return result


a = [1, 2, 3]       # 1 + 2x + 3x^2
b = [4, 5]          # 4 + 5x

assert naive_convolution(a, b) == [4, 13, 22, 15]
```

La traza corresponde a:

$$\begin{aligned} c_0 &= 1(4) = 4, \\ c_1 &= 1(5) + 2(4) = 13, \\ c_2 &= 2(5) + 3(4) = 22, \\ c_3 &= 3(5) = 15. \end{aligned}$$

El costo es $\Theta(mn)$ tiempo y $\Theta(m + n)$ memoria para la salida.

8.2. Ejercicio resuelto 2: FFT, IFFT y multiplicación

Listing 3: FFT recursiva e IFFT

```python
from cmath import exp, pi


def fft(a):
    n = len(a)
    if n == 1:
        return list(a)
    if n == 0 or n & (n - 1):
        raise ValueError("La longitud debe ser potencia de dos")

    even = fft(a[0::2])
    odd = fft(a[1::2])
    root = exp(2j * pi / n)
    w = 1
    result = [0j] * n

    for k in range(n // 2):
        t = w * odd[k]
        result[k] = even[k] + t
        result[k + n // 2] = even[k] - t
        w *= root
    return result
```

```
23
24
25 def ifft(values):
26     n = len(values)
27     transformed = fft([z.conjugate() for z in values])
28     return [z.conjugate() / n for z in transformed]
```

Listing 4: Multiplicación de polinomios mediante FFT

```
1  def next_power_of_two(value):
2      n = 1
3      while n < value:
4          n <<= 1
5      return n
6
7
8  def multiply_fft(a, b, tolerance=1e-7):
9      if not a or not b:
10         return []
11
12     output_length = len(a) + len(b) - 1
13     n = next_power_of_two(output_length)
14     pa = list(a) + [0] * (n - len(a))
15     pb = list(b) + [0] * (n - len(b))
16
17     fa, fb = fft(pa), fft(pb)
18     fc = [x * y for x, y in zip(fa, fb)]
19     values = ifft(fc)[:output_length]
20
21     result = []
22     for z in values:
23         if abs(z.imag) > tolerance:
24             raise ArithmeticError("Error imaginario excesivo")
25         result.append(round(z.real))
26     return result
27
28
29 assert multiply_fft([1, 2, 3], [4, 5]) == [4, 13, 22, 15]
```

La longitud mínima es $3 + 2 - 1 = 4$, que ya es potencia de dos. La FFT evalúa ambos polinomios en $1, i, -1, -i$, se multiplican los valores y la IFFT recupera los coeficientes.

8.3. Ejercicio resuelto 3: NTT e INTT en $\mathbb{F}_{17}$

Para longitud 4, el elemento $\omega = 4$ es una raíz cuarta primitiva módulo 17 porque $4^2 \equiv -1$ y $4^4 \equiv 1$ (mód 17).

Listing 5: NTT directa de referencia

```
1  def ntt_direct(values, root, modulus):
2      n = len(values)
3      result = []
4      for k in range(n):
5          total = 0
6          for j, value in enumerate(values):
7              total += value * pow(root, j * k, modulus)
8          result.append(total % modulus)
9      return result
10
11
```

```python
def intt_direct(values, root, modulus):
    n = len(values)
    root_inverse = pow(root, -1, modulus)
    n_inverse = pow(n, -1, modulus)
    transformed = ntt_direct(values, root_inverse, modulus)
    return [(x * n_inverse) % modulus for x in transformed]


a = [1, 2, 3, 4]
A = ntt_direct(a, root=4, modulus=17)
assert A == [10, 7, 15, 6]
assert intt_direct(A, root=4, modulus=17) == a
```

Objetivo del ejercicio resuelto

Esta versión directa cuesta $\Theta(N^2)$ y sirve como oráculo de referencia. En el ejercicio propuesto se implementará la NTT radix-2 de $\Theta(N \log N)$.

9. Ejercicios propuestos

9.1. Ejercicio propuesto 1: biblioteca integrada de FFT y NTT

Construya una biblioteca denominada `transforms.py`. Debe contener:

1. convolución directa;
2. FFT e IFFT recursivas;
3. FFT iterativa in-place con inversión de bits;
4. multiplicación mediante FFT con tolerancia configurable;
5. validación del orden de una raíz modular;
6. NTT e INTT radix-2;
7. multiplicación mediante NTT;
8. contadores de mariposas, multiplicaciones, sumas y etapas.

Requisitos funcionales:

1. rechazar longitudes no compatibles;
2. elegir automáticamente la potencia de dos para FFT;
3. para NTT, comprobar $N \mid (p-1)$ y que la raíz tenga orden N;
4. devolver exactamente $m+n-1$ coeficientes;
5. aceptar coeficientes negativos mediante normalización modular;
6. comparar los resultados con la convolución directa;
7. documentar precondiciones, poscondiciones y excepciones.

Para la NTT puede utilizar inicialmente $p = 17$ con longitudes hasta 16, y posteriormente un primo adecuado para tamaños mayores. El equipo deberá justificar el primo, la raíz primitiva y las longitudes admitidas.

9.2. Ejercicio propuesto 2: pruebas diferenciales y análisis experimental

Compare convolución directa, FFT recursiva, FFT iterativa y NTT. Use vectores de longitudes:

$$2^k, \qquad k = 3, 4, \ldots, 14,$$

siempre que la memoria y el tiempo disponibles lo permitan.

Escenario A: corrección

1. genere coeficientes enteros reproducibles en rangos $[-10, 10]$, $[-10^3, 10^3]$ y $[0, p-1]$;
2. compare cada método con la convolución directa para tamaños pequeños;
3. compare FFT recursiva e iterativa para todos los tamaños;
4. compare NTT con la convolución módulo p;
5. registre semilla, entrada y salida ante cualquier discrepancia.

Escenario B: rendimiento

1. mida tiempo mediano, mínimo y máximo de al menos 10 repeticiones;
2. registre operaciones y etapas;
3. determine desde qué tamaño FFT o NTT supera al método directo;
4. compare recursión frente a versión iterativa;
5. interprete resultados considerando constantes, caché, objetos complejos y aritmética modular.

Escenario C: exactitud

1. incremente gradualmente tamaño y magnitud de coeficientes;
2. registre el máximo error real e imaginario antes del redondeo;
3. identifique el primer caso donde FFT no coincide exactamente con la referencia;
4. compruebe que NTT permanece exacta módulo p mientras no exista ambigüedad por reducción modular.

Interpretación obligatoria

La NTT entrega coeficientes módulo p. Para recuperar coeficientes enteros sin ambigüedad, el módulo debe superar el rango posible de los coeficientes o deben combinarse varios módulos mediante el teorema chino del resto.

10. Metodología de medición

Listing 6: Medición reproducible

```python
from statistics import median
from time import perf_counter_ns


def measure(function, repetitions=10):
    times = []
```

```
    for _ in range(repetitions):
        start = perf_counter_ns()
        function()
        times.append(perf_counter_ns() - start)
    return {
        "min_ns": min(times),
        "median_ns": median(times),
        "max_ns": max(times),
    }
```

Para evitar resultados engañosos:

- genere las entradas antes de medir;
- use los mismos vectores para todos los métodos;
- reconstruya entradas cuando un método opere in-place;
- no incluya impresiones ni escritura de archivos;
- verifique primero la corrección;
- registre versión de Python, sistema operativo y características básicas del equipo;
- utilice semillas fijas en los experimentos.

11. Resultados mínimos requeridos

Los resultados mínimos son:

1. tabla de casos manuales y resultados obtenidos;
2. validación de FFT seguida de IFFT;
3. validación de NTT seguida de INTT;
4. gráfico de tiempo frente a tamaño para los cuatro métodos;
5. gráfico de mariposas u operaciones frente a N;
6. tabla del máximo error numérico de FFT;
7. identificación del punto de cruce con el método directo;
8. tabla de módulos, raíces y longitudes válidas para NTT;
9. archivo CSV con parámetros, semillas y mediciones.

12. Preguntas de análisis

Responda sustentando con resultados:

1. ¿Por qué la multiplicación de polinomios es una convolución?
2. ¿Por qué se requieren $m + n - 1$ posiciones?
3. ¿Qué diferencia existe entre convolución lineal y circular?

4. ¿Qué propiedad de las raíces permite dividir el problema en pares e impares?
5. ¿Qué trabajo realiza una mariposa?
6. ¿Por qué la FFT es $\Theta(N \log N)$?
7. ¿Qué diferencias prácticas aparecen entre FFT recursiva e iterativa?
8. ¿Cuándo aparecen errores de redondeo relevantes?
9. ¿Por qué una NTT necesita que $N \mid (p-1)$?
10. ¿Qué diferencia existe entre exactitud modular y exactitud entera?
11. ¿Cuándo conviene FFT compleja y cuándo NTT?
12. ¿Por qué una mejor complejidad asintótica puede no ser más rápida para tamaños pequeños?

13. Producto de entrega

Producto de entrega

El equipo entregará:

1. código completo de los ejercicios resueltos y propuestos;
2. biblioteca `transforms.py`;
3. pruebas automatizadas y generador reproducible;
4. archivos CSV y gráficos solicitados;
5. informe PDF con metodología, resultados y respuestas;
6. conclusiones grupales de 250 a 350 palabras.

14. Tabla de evaluación

Criterio	Indicador de logro	Puntaje
Asistencia y participación en el laboratorio	Asiste, permanece durante la práctica y participa en implementación, pruebas, análisis y revisiones. Se determina individualmente.	4
Corrección y funcionamiento	Convolución, FFT, IFFT, NTT e INTT funcionan; las pruebas comprueban padding, raíces, inversión y equivalencia con las referencias.	8
Análisis de complejidad y resultados	Relaciona tamaño, operaciones, tiempo, memoria y error numérico; interpreta correctamente FFT frente a NTT y método directo.	3
Dominio individual	Explica una mariposa, ejecuta una traza, valida una raíz o modifica una prueba durante la revisión.	4
Total		**19**

15. Casos mínimos de prueba

- vectores vacíos y de un solo coeficiente;
- polinomio cero y constante;
- longitudes iguales y diferentes;
- coeficientes positivos, negativos y cero;
- longitud exacta y longitud que requiere padding;
- FFT de longitudes 2, 4, 8 y 16;
- rechazo de longitud no potencia de dos;
- raíces modulares válidas e inválidas;
- coeficientes cercanos al módulo;
- casos aleatorios comparados con la convolución directa;
- tamaños y coeficientes que revelen error flotante.

16. Referencias

Referencias

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest y C. Stein. *Introduction to Algorithms*, 4.ª ed. MIT Press, 2022.

[2] E. O. Brigham. *The Fast Fourier Transform and Its Applications*. Prentice Hall, 1988.

[3] J. von zur Gathen y J. Gerhard. *Modern Computer Algebra*, 3.ª ed. Cambridge University Press, 2013.

[4] J. W. Cooley y J. W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 1965.

[5] M. Frigo y S. G. Johnson. The Design and Implementation of FFTW3. *Proceedings of the IEEE*, 2005.